Township: Technical Design Document

Repository	https://github.com/AmanSingh-tech/township
Live app	https://township-91vm.onrender.com/welcome. Demo sign-ins are listed on the sign-in page; codes appear on screen because SMS is mocked. Running it locally and every login are in deployment.md.
Client	Company A, Pune: a gated township of 2,400 flats in 28 towers
Author, date	Aman Kumar Singh, 5 November 2026

How to read this: Part A is what was built and why, Part B is how it works inside (low-level design, concurrency, scaling), Part C answers the questions a reviewer is likely to ask. Every assumption is labelled A1–A12 in section 3.

0. The assignment, point by point

The assignment asks for	Where it is met
The core build working end to end	All 8 core screens on the brief's sample data (section 2.1); the brief's three flats reproduce its table exactly, and a test checks them
Main flows on realistic sample data; no dead buttons	A sample township (the brief's 3 flats plus about 60 more, with bills, payments, complaints, visitors, bookings and notices) loads on first start; 6 browser tests click through the main flows and fail on any page error
Frontend, backend, database, sign-in, roles	React app, Node and Express API, PostgreSQL; one-time-code sign-in; 6 roles, checked on every route (section 9)
The AI part works, including when wrong or unsure	Claude or a local model for category and urgency; a safety rule above both; "unsure" flagged below 60% confidence; one-tap correction; measured on 90 labelled complaints (section 7)
Polished UI: consistent, works on a phone, loading, empty and error states	One design system for phone, desk, gate tablet and front page; phone-first with a desktop layout; shared skeleton, empty and error components (section 8)
Tests for the key logic; no secrets; validated inputs; access control	53 server and 6 browser tests, an evaluation suite, a load test (section 10); only `.env.example` is committed; every route validates input and declares its roles (section 9)
TDD: GitHub URL, live link, architecture and why, data model, flows, AI, UI decisions, cuts and next	This document: sections 4, 5, 6, 7, 8 and 14
deployment.md: run locally, how it is deployed, environment variable names, demo logins	deployment.md, sections 1–6
Use only the brief; label every assumption	Section 3 separates what the brief gives from A1–A12
Keep the commit history; no secrets or real personal data	Every change is its own described commit; all names, numbers and UPI IDs in the sample data are invented

PART A · WHAT WAS BUILT, AND WHY

1. The client's problem, and how the product answers it

Bills are made in Excel and emailed, payments are matched by hand before Tally, complaints arrive by phone and 28 WhatsApp groups, visitors go into paper registers at 4 gates, and the clubhouse is booked on a spreadsheet. The brief costs this at **₹83.57 lakh a year** and sets three six-month targets. Each target drove a design decision:

Brief's cost and target	What in the product addresses it	Where the committee sees it
₹64.8 lakh never collected; on-time payments 70% → 90%	Dues are the first thing on the home screen, with the reason; paying takes two taps; residents are alerted when bills issue; interest is shown, never a surprise	"Paid by due date" against 90%, and dues ageing
₹8.64 lakh of staff time matching; matching time −80%	Credits with a flat number or registered UPI ID post themselves; the rest arrive sorted, with evidence and the three best flats; re-uploads are harmless; UPI IDs are learned	"Received, not yet identified"; the evaluation measures 88% less time (section 7)
₹10.13 lakh of reopened complaints; reopened 25% → under 8%	The right technician from the start, a response timer, every step visible to the resident, duplicates linked into one job, closure only with the resident's rating	"Reopened" against 8%, first response and resolution times

2. Scope

2.1 The core build: 8 screens, end to end

Screen	Role	What it does
Sign in	All	Mobile number and a one-time code, linked to a flat (owner, tenant or family)
Resident home	Resident	Dues with a breakdown, open complaints and today's visitors; every action within two taps
Bills and test payment	Resident	Area × rate, late interest, a six-month chart, test-mode checkout, numbered receipt with PDF, ledger
Raise a complaint	Resident	Text and photo; AI category and urgency; safety rule
Complaint status	Resident	Timeline, assignee, response timer; rate the fix or reopen
Help desk queue	Help desk	Sorted by safety, urgency and deadline; auto-assignment; one-tap category change; linked duplicates
Bank matching	Accounts	The hard part: bank CSV in; confident credits post, unclear ones queue with evidence
Committee report	Committee	Collections, dues ageing, complaint times, the brief's targets; signed and frozen monthly

2.2 Every "must have" from the brief

Must have	How it is built
Installable web app; OTP login linked to a flat; family and tenant members; notices	A web app (manifest, service worker, PNG and maskable icons, shortcuts) with a Download app button: the browser's install prompt on Chrome and Edge, the steps on Safari, hidden once installed. Owners and tenants add household members; notices on the home screen

Billing by area and rate; late fees per bye-laws; online payment (test mode); bank matching; receipts and a ledger per flat	Monthly bill runs (area × rate copied at issue), simple interest on overdue principal (A2), a test gateway that follows Razorpay's steps, the matcher (section 7), gapless receipts with PDFs, an append-only ledger per flat
Complaints: category, photo, auto-assignment, response timer, status updates, rating on closure	All of these; technicians by category and tower; response targets per urgency (A6); the resident rates or reopens
Gate: tablet entry with photo, resident approval, pre-approved guest codes	A paired tablet with a guard PIN, photo capture, a push to the flat, guest codes; an offline queue
Amenity booking with slots, rules and fees	Slots and capacity, per-booking and weekly limits, booking window, fees on the bill, free cancellation until a cut-off, closed days that refund
Committee dashboard: collections, dues ageing, complaint times	The committee report, with the brief's targets, CSV and print, and a signed, fingerprinted month

2.3 The brief's constraints and design asks

The brief says	How it is met
DPDP Act: delete visitor photos after 90 days; a notice at gates	A scheduled job deletes them; the notice is on the gate tablet and printable for each gate; consent at first sign-in, data download, correction and erasure
Billing and late interest per Maharashtra co-operative bye-laws	Rate, due day and interest rate are settings; A2 and A3 state the reading
Gate tablets keep working through network drops	Entries queue on the tablet with a client id and sync when the network returns; the server ignores repeats (section 12)
English, Marathi and Hindi	All three in the resident app and gate tablet; the safety rule and classifier read all three
Most bills are paid in the first 10 days	Due on the 10th (A3); the load test is sized from that peak
Minimal design; home shows dues, open complaints, today's visitors; two taps; guard screens with very large buttons that read at night	Section 8
Out of scope: accounting beyond maintenance, CCTV and barriers, a marketplace	Not built; Tally stays, and the app exports a Tally import file

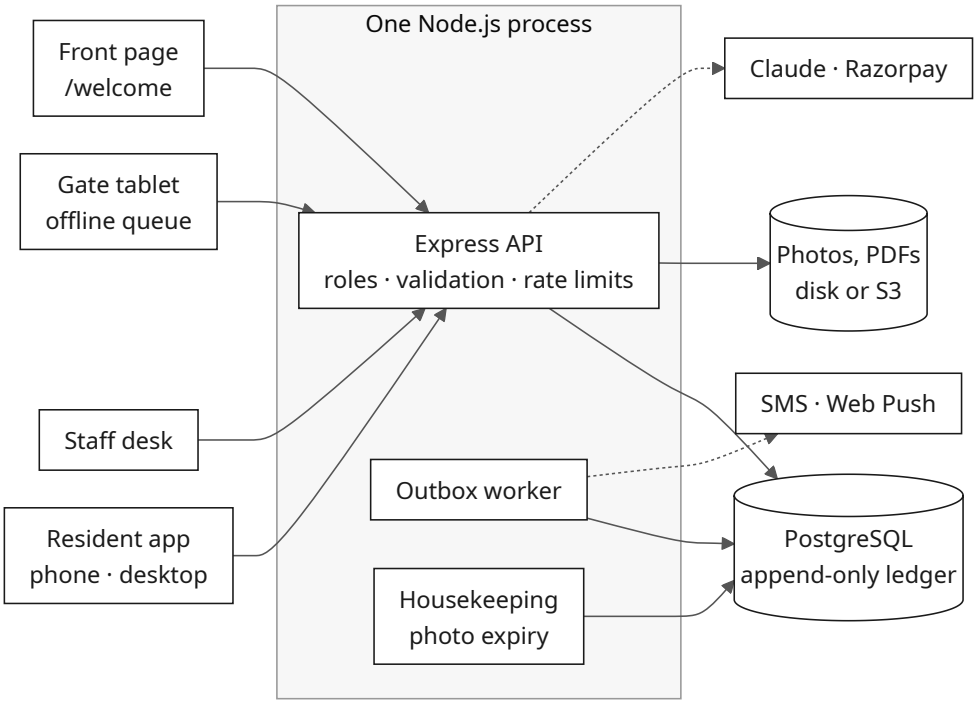
3. Assumptions

Given by the brief, so not assumptions: ₹6.50 per sq ft a month; the three sample flats; late fees under the bye-laws; visitor photos deleted after 90 days with a gate notice; English, Marathi and Hindi; mocked payments and messages; most bills paid in the first 10 days. Where the brief is silent, I chose, and each choice is a setting or one function:

#	Assumption	Why

A1	"Days overdue" counts from the oldest unpaid due date, so T12-0408 (47 days) also owes the next month.	The only reading where the sample row and a monthly bill agree
A2	Late fee means simple interest on overdue principal at 21% a year; payments clear interest first, then the oldest bill.	21% is the model bye-law ceiling; interest on interest would be compounding
A3	Bills are raised on the 1st and due on the 10th; opening arrears carry no interest in the app.	Matches the payment pattern; old arrears' interest cannot be recomputed reliably
A4	Bank statements are CSV exports; UPI narrations carry the payer name, UPI ID and UTR, sometimes a flat number.	How HDFC, SBI and ICICI exports look; columns are found by heading
A5	A credit posts automatically only on a flat number or registered UPI ID, a score of 80+ and a lead of 30 over the next flat.	The brief: unclear payments are never posted automatically
A6	Response targets: 15 min critical, 2 h high, 8 h medium, 24 h low; one duty-manager number.	A starting point the admin can change
A7	Roles beyond the core two: accounts, committee, admin, guard.	The other must-haves, and "the committee approves", need them
A8	Money decisions need two different people (bill runs, manual posts, adjustments, login-number changes).	What makes a report "trustworthy" to a committee
A9	Receipts are numbered without gaps per financial year: `RCT/2026-27/00001`.	Indian practice; auditors look for gaps
A10	Each of the 4 gates has one tablet, paired once; guards share tablets across shifts and sign in with a PIN.	Paper registers today mean one point of entry per gate
A11	One server is enough for one township: about 8 requests a second at the bill-day peak.	2,400 flats, most paying in 10 days (section 13)
A12	For the matching evaluation: the mix of payment styles, and 2 minutes to match a row by hand, 30 seconds to confirm a right suggestion.	Stated so the 88% can be checked and re-run with other numbers

4. Architecture (high-level design)



React and Vite on the client; Node 20, Express and PostgreSQL 16 on the server; one Docker image, deployed on Render. Dotted lines are paid services, replaced by labelled placeholders when no credentials are set.

The shape follows from the numbers: one township, about 8 requests a second at peak, money that must never be wrong, and a team of one. A single process with clear modules and one database gives transactions across everything that must change together (a payment, its ledger line, its receipt number and its alert), one thing to deploy, and nothing to keep in sync. The decisions, in the usual ADR form:

> **ADR-1 · One modular service, not microservices**
>
> **Context:** one township peaks at about 8 requests a second. **Decision:** one Node process with modules (billing, matcher, complaints, notify, amenities, report). **Consequences:** one deploy and no network between services; the load test reaches 270–300 requests a second on one process, and the app is stateless, so more processes can be added (section 13).

> **ADR-2 · PostgreSQL, with the ledger append-only in the database**
>
> **Context:** the committee needs a report it can trust. **Decision:** bills and payments write ledger lines in one transaction; triggers block updates and deletes; the app's database role cannot switch them off. **Consequences:** mistakes are corrected by new adjustment lines; `cli verify` proves the books balance and receipts have no gaps.

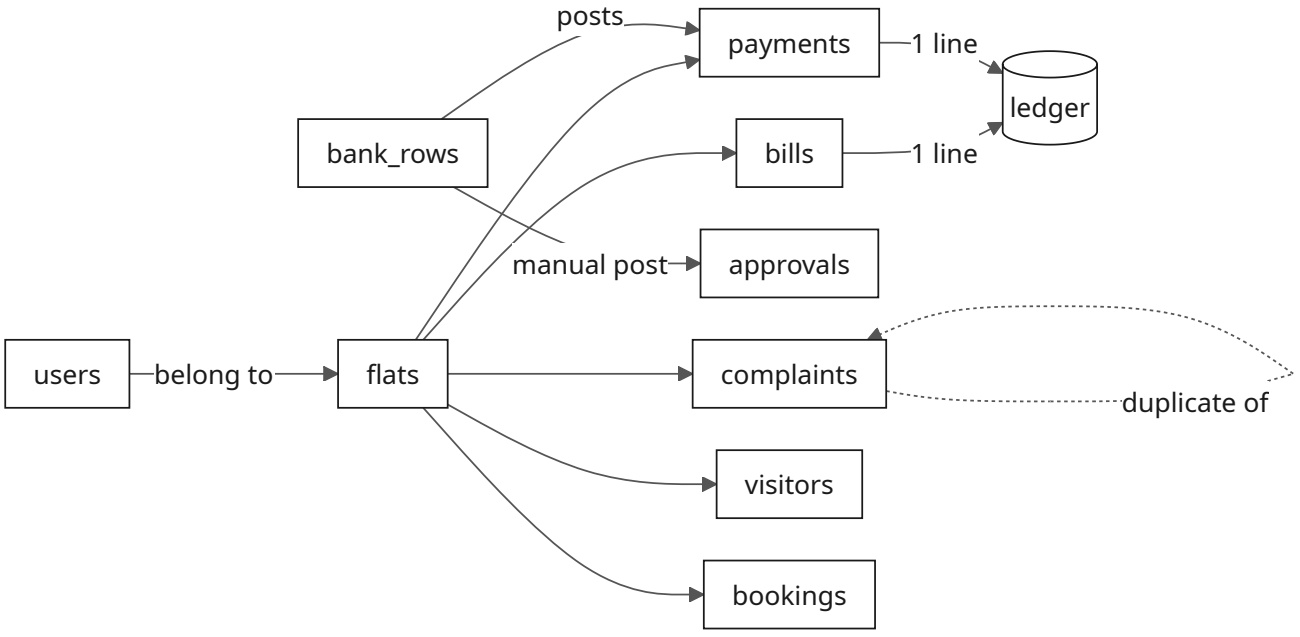
> **ADR-3 · Payment matching by explained rules, not a model**
>
> **Context:** a wrong match puts money on the wrong flat. **Decision:** score each credit with reasons; post only on strong, unambiguous evidence; queue the rest for a person, with a second person approving. **Consequences:** every post is repeatable and auditable; no wrong posts in the evaluation.

> **ADR-4 · Complaint triage by LLM, with a rule above it and a local fallback**
>
> **Context:** a wrong category costs a tap; a missed gas leak does not bear thinking about. **Decision:** safety words escalate by rule first; Claude classifies with a fixed schema; a local keyword model runs without a key or on any error. **Consequences:** a complaint is never blocked by the network, and the AI is never the last word on safety.

> **ADR-5 · Messages through a database outbox**
>
> **Context:** a crash must not lose a visitor alert, or send one for a payment that rolled back. **Decision:** messages are written in the same transaction as their cause and sent by a worker, urgent first. **Consequences:** no message queue to run; a visitor alert behind 4,200 bill alerts goes out in 0.9 s.

> **ADR-6 · An installable web app with one-time-code sign-in**
>
> **Context:** residents open the app for seconds and forget passwords; the brief asks for an installable web app. **Decision:** one web app installable on phones and desktops, with push alerts; codes to the registered number; guards use a PIN on a paired tablet. **Consequences:** no app store, one codebase for every screen size and the gate tablet.

5. Data model



34 tables in five migrations; the core is shown. A flat's balance is always the sum of its ledger, never a stored number that could drift. Also: bill runs, receipt counters, payment orders, bank imports, complaint events, technicians, guest codes, devices, sessions, OTPs, messages (the outbox), audit log, report sign-offs, amenities and blackouts, register imports, Tally exports, data requests, push subscriptions, notices, settings.

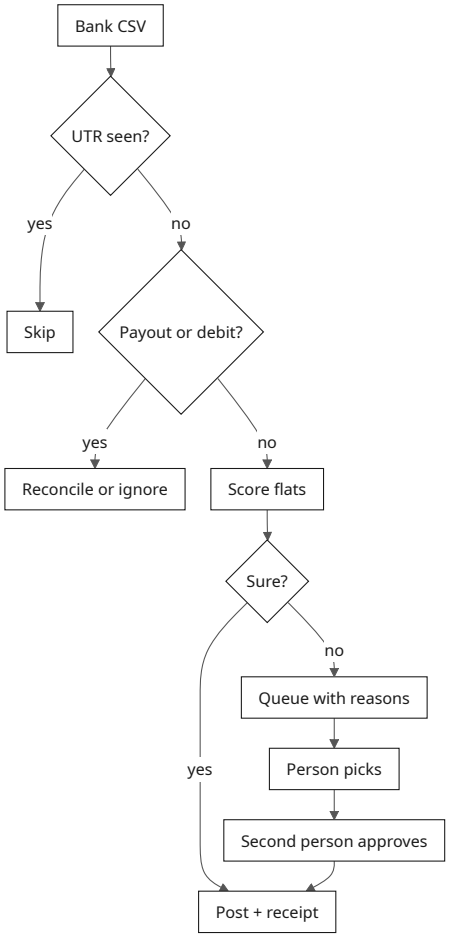
Table	Columns that matter, and why
flats	`id` like `T12-0408` (checked by pattern), `tower`, `area`, which drives the bill
users	`phone` (unique; the login), `role`, `flat_id`, `relation` (owner, tenant, family), `lang`, `pin_hash` for guards
bills	`period`, `area` and `rate` copied at issue (a later rate change never rewrites an old bill), `amount`, `due_on`; unique per flat and month

payments	`amount`, `method`, `utr`, `gateway_payment_id` (unique), `receipt_no` (unique, gapless)
ledger	`kind` (bill, interest, payment, amenity, adjustment, opening), `debit`/`credit` in whole rupees, a link to what it records
bank_rows	`narration`, `utr`, `status` (posted, queue, pending approval, duplicate, reconciled, ignored), `score`, `reasons` shown to the treasurer
complaints	`category`, `urgency`, the AI's own `ai_category`/`ai_confidence`/`ai_source` (kept to measure it), `escalated`, `duplicate_of`, `due_by`
visitors, bookings	Visitor `status` and a unique `client_id` from the tablet; booking slot, `fee` and `status`

Rules the database enforces, not only the app: triggers make `ledger`, `payments`, `audit_log` and `report_signoffs` append-only; `UNIQUE (flat_id, period)` stops double billing; `UNIQUE gateway_payment_id` records a payment once; `CHECK (decided_by <> requested_by)` stops self-approval; money is integer rupees with non-negative checks; phones and flat ids are checked by pattern.

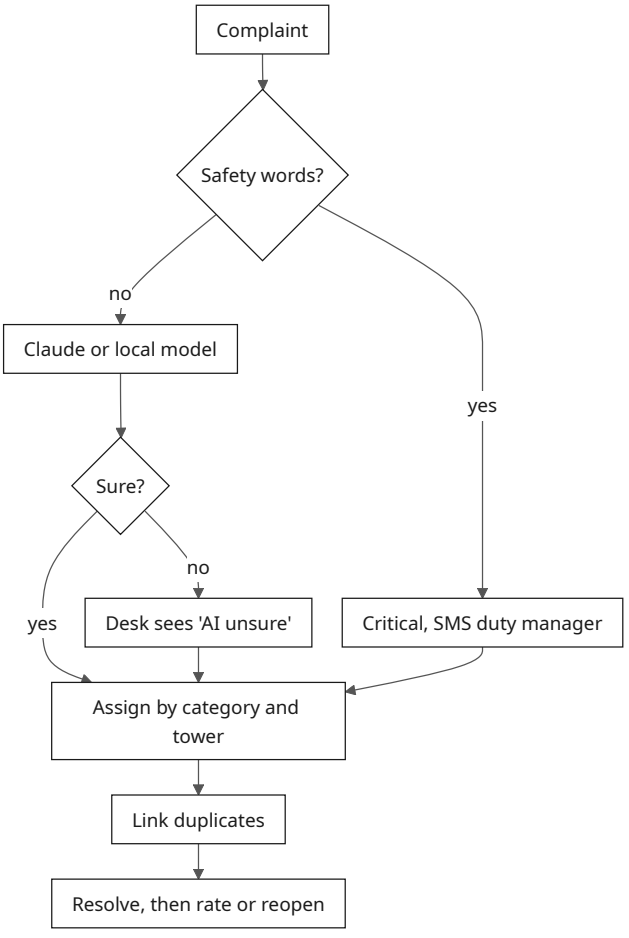
6. Main flows

Bank statement matching (the hard part)



"Sure" means a flat number or registered UPI ID, a score of 80+, a lead of 30+, and no other flat with strong evidence.

Complaint triage



"Sure" is confidence of 0.6 or more. Safety words: fire, gas, lift stuck, shock, in three languages.

Payment: the server creates the gateway order for the amount it computes (never the browser's); after checkout it checks the signature and fetches the payment from the gateway; then the payment, ledger line,

receipt number and alert are written in one transaction. A signed webhook covers residents who close the app mid-payment, and either path records the payment exactly once. **Bill run:** accounts prepares a month's bills (a draft); a committee member who did not prepare it approves; only then are bills issued, at the draft's rate, and residents alerted. **Gate:** the guard logs a visitor; the flat gets a push; the resident approves or denies; offline, entries wait on the tablet and sync later.

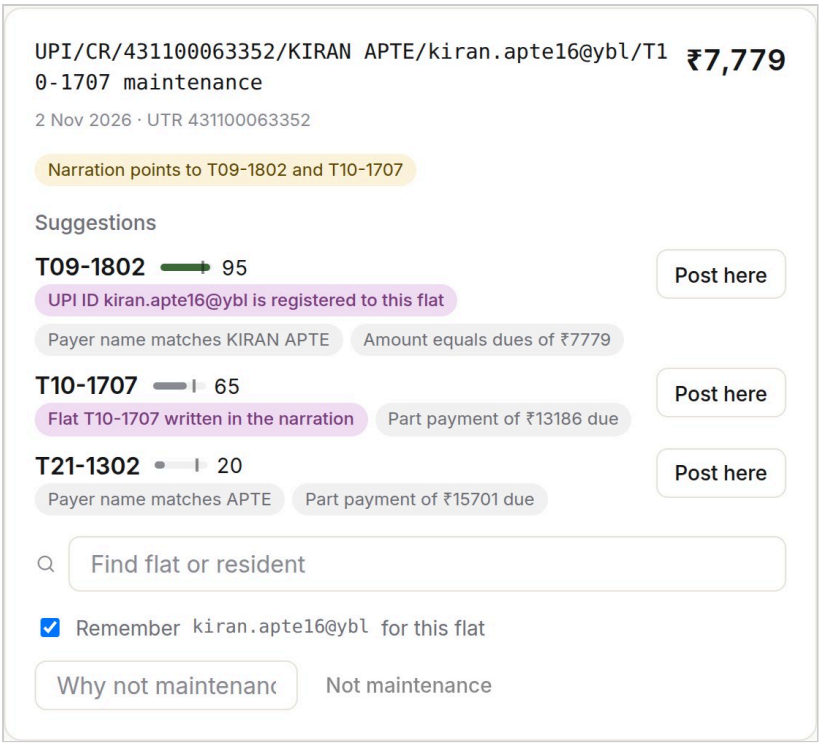
7. The AI and automation

The brief requires	Built as
Match bank rows by amount, reference and UPI ID	An evidence score: flat number in the reference +60, registered UPI ID +50, amount equals dues +30 (one month's bill +20), payer name +15, part payment +5; far above dues −40
Unclear payments go to a queue, never posted automatically	Name or amount alone never posts; rival strong evidence queues; a manual post needs a second person
Classify complaints by category and urgency	Claude with a fixed category list and a JSON schema; refusals, invalid answers or an 8 s timeout fall back to the local model
The help desk changes a category in one tap	A category menu; below 0.6 confidence the complaint is flagged "AI unsure"; the AI's guess is kept
Fire, gas, lift stuck, shock always escalate	A rule before the AI, in English, Hindi and Marathi; critical urgency that cannot be lowered; SMS to the duty manager
Link duplicates, never close them	Same category and tower within 48 hours, about a shared asset or in similar words: linked to the main complaint, whose updates reach every flat; each flat closes and rates its own

Measured, not assumed (`npm run eval`, docs/evaluation.md)

Complaint triage: 90 labelled complaints in English, Hindi, Marathi, Hinglish	**Local model**
Category right; right when the model says it is sure	81%; 100% (for 35% of complaints)
Flagged "AI unsure" for the help desk to check	64%
Hazards escalated; false alarms	14 of 14; 0
Bank matching: 5 statements, 140 credits with known flats, 30 other rows	**Result**
Posted automatically; **posted to the wrong flat**	55%; **0**
Queued with the right flat as the top suggestion	98% (100% in the top three)
Matching time against doing it by hand (target −80%)	−88% (A12)

Honest confidence. The local model reports the share of matched keywords discounted by $1 - 0.5^n$, so one keyword gives about 50%, not 100%. **What the first evaluation found and fixed:** Hindi and Marathi words were split on their vowel signs, so their keywords never matched (50% right; now 86% and 71%); the Marathi "अडकले" (stuck) was missed; "लिफ्ट बंद" (the lift is off) and "fire lane" raised false alarms. The labels follow the urgency rules given to the model, not its keyword list, so the score is not rigged; a test keeps the key promises as floors.

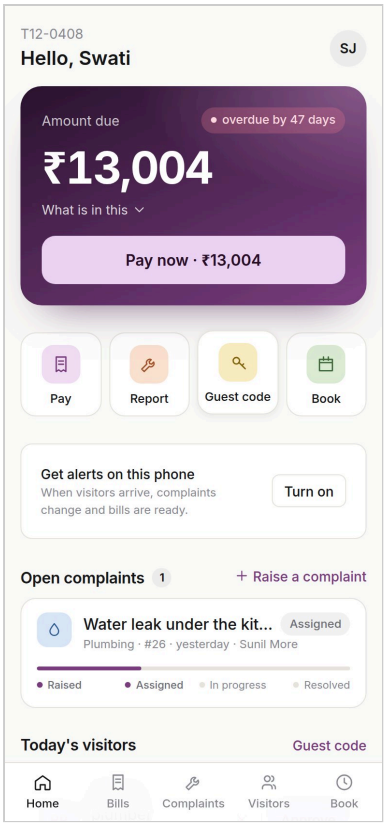


A credit whose narration names T10-1707 while its UPI ID is registered to T09-1802: two strong signals disagree, so it is queued with both flats and their evidence instead of being posted.

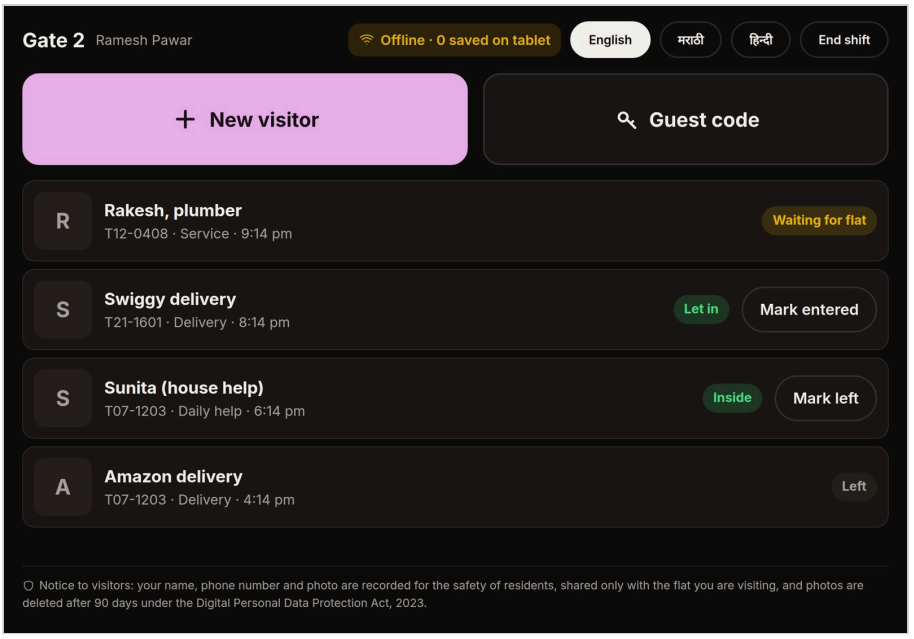
8. UI and design decisions

The brief asks for minimal design that counts as much as features: clean layouts, white space, a restrained palette, clear type, nothing without a purpose, every action within two taps. The thinking behind each choice:

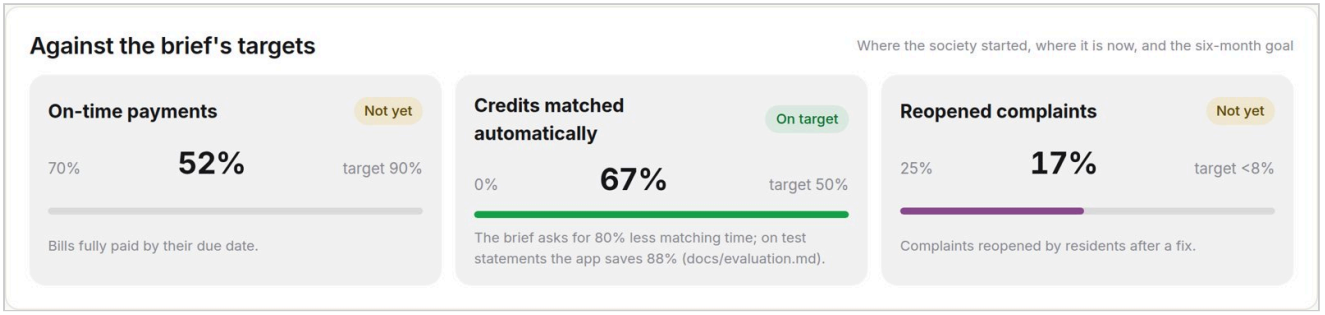
- **One accent, used for the next action.** Warm neutrals with a lilac button (dark text on it) and plum for links; six soft tints only to tell categories, people and amenities apart; green, amber and red reserved for state. The eye always finds the one thing to do.
- **Type for moments, not decoration.** A display serif (Instrument Serif) carries greetings, page titles and big amounts; everything else is Inter for reading.
- **The home screen is the brief's sentence:** dues (with "What is in this" for the breakdown), four quick actions (pay, report, guest code, book), open complaints with named steps, today's visitors with approve in one tap.
- **Small interactions that explain:** a tapped month in Bills shows its sum (980 sq ft × ₹6.50); a waiting visitor pulses on a timeline; a guest code counts down; picked booking slots join into one block. CSS only, and off when the phone asks for reduced motion.
- **Phone first, desktop proper:** bottom tabs on phones; from 960 px a sidebar and a two-column home. The desk is denser, for a working day; the committee report leads with the brief's targets.
- **The gate is for night and gloves:** always dark, very large buttons, state in colour and words, "Offline · saved on tablet", the DPDP notice on screen.
- **Light and dark themes:** dark is a warm plum night, not flat black. **States:** skeletons while loading, written empty states, errors that say what to do. **Three languages** throughout the resident app and gate.
- **A front page** (`/welcome`) tells the product in seven chapters, each its own colour, arriving as sheets; it loads on its own, so the app carries none of its weight. **Logo:** a T made from a tower between two shorter ones.



Resident home: dues first, with overdue days and interest stated.



Gate tablet with the network down: large buttons, dark for night, the offline state, the DPDP notice.



Committee report: the brief's success measures from where the society started, against the six-month targets.

9. Security and privacy

- **Sign-in:** codes are random, stored as hashes, valid for 5 minutes with 5 tries; at most 3 per number an hour (15 in demo mode) and limits per network address. Unknown numbers get the same reply as known ones, so the endpoint cannot reveal who lives here.
- **Sessions:** random tokens stored as hashes; 30 days for residents, 12 hours for staff and guards; deactivating a person ends them.
- **Access:** every route declares its roles; a resident only ever reaches their own flat (every query is scoped by the session's flat); the gate needs a paired tablet and a PIN.
- **Inputs and outputs:** every route validates its body; CSV exports neutralise spreadsheet formulas; uploads must be real images; a strict Content Security Policy with fonts bundled, so the only third party is the payment gateway.
- **Secrets and data:** only `.env.example` is committed; the server refuses to start in production without its required settings. Photos are served through 15-minute signed links. DPDP: consent, data download, correction, erasure (accounts kept as the law requires), visitor photos deleted after 90 days.

10. Testing and evaluation

- **53 server tests:** the brief's sample flats, interest, payment allocation, gapless receipts, append-only enforcement, every matching case, safety words in three languages, duplicate linking, role access, two-person approvals, the gate, privacy, and the evaluation floors (every hazard caught, no false alarms, no wrong posts, 80% time saved).
- **6 browser tests** in headless Chrome: a complaint raised, resolved and rated; a manual post approved by a second person; bills prepared, approved and the report signed; a booking charged; a guard's visitor approved from the phone; no browser errors on any page.
- **Load test at full size** (2,400 flats, a year of data, 393,000 gate entries): 95% of requests under 28 ms at five times the bill-day peak; a bill run for every flat in 4.1 s; 2,400 bank credits in 17.5 s; one process tops out at 270–300 requests a second.

PART B · HOW IT WORKS INSIDE

11. Low-level design

11.1 Modules

Where	What it owns
`server/routes/`	HTTP only: `auth`, `resident`, `desk`, `gate`, `admin`. Each route declares its roles and validates its input, then calls a library function
`server/lib/`	The rules: `billing` (bills, interest, allocation, receipts), `billruns`, `matcher` and `csv`, `approvals`, `complaints` and `classifier`, `amenities`, `report`, `notify` (outbox), `auth`, `privacy`, `audit`, `tally`, `receiptPdf`, `storage`
`server/db.js`, `migrations/`	A connection pool, `tx()` (nested calls join the outer transaction), versioned SQL migrations
`server/eval/`, `test/`	The labelled sets and evaluators; the test suites
`web/src/`	`pages/resident`, `pages/desk`, `pages/gate`, `landing/` (front page), shared `ui.jsx`, `api.js`, `i18n.jsx`, `install.js`

11.2 A request's life

Security headers → JSON or CSV body limits → session lookup (hash of the bearer token; a paired tablet adds a device token) → role check for the route → input validation → the library function, inside `tx()` when it changes data → audit entry and outbox messages in the same transaction → JSON response. Errors become a plain message and a status code; nothing internal leaks.

11.3 Key algorithms

- **Dues and interest:** interest accrues daily on each bill's unpaid principal after its due date, at `rate × days / 365`, rounded to whole rupees; it is posted as a ledger line before any payment, so a payment always clears interest first, then the oldest bill.
- **Receipt numbers:** one counter row per financial year, incremented in the same statement that inserts the payment, so numbers are gapless even under concurrency.
- **Matching:** extract the UTR, UPI ID, flat references in the forms residents write them and name words; score candidate flats; decide (A5). A whole statement is matched against an in-memory index of flats, UPI IDs and dues, refreshed after each post.

- **Duplicate complaints:** an open complaint in the same category and tower from the last 48 hours, about a shared asset (such as a lift) or with word overlap of 0.3 or more → linked to the main one.
- **Committee sign-off:** the month's figures are frozen as JSON with a SHA-256 fingerprint, re-checked every time the signed copy is shown.

11.4 States

Thing	States
Complaint	open → assigned → in progress → resolved → closed (rated) · a reopen sends a resolved complaint back to work and is counted
Bank row	posted · queue → pending approval → resolved · duplicate · reconciled · ignored
Bill run	draft → issued (when a second person approves) · draft → rejected
Visitor	waiting → approved or denied → inside → left

11.5 API, by area

Auth: `POST /auth/otp`, `/auth/verify`, `/auth/pin`, `/devices/pair`. **Resident:** `/home`, `/bills`, `/pay/order`, `/pay/verify`, `/receipts/:id/pdf`, `/complaints` (+ `rate`, `reopen`), `/visitors/:id/decide`, `/guest-codes`, `/amenities/:id/availability`, `/bookings`, `/household`, `/me/export`. **Desk:** `/desk/complaints`, `/desk/bank/import`, `/desk/bank/queue`, `/desk/bank/rows/:id/post`, `/desk/approvals/:id/approve`, `/desk/bill-runs`, `/desk/report`, `/desk/tally/export`, `/desk/register/preview`. **Gate:** `/gate/visitors`, `/gate/code`. **Admin:** users, devices, staff, settings, notices, amenities, data requests, audit, messages. **Payments:** `/api/payments/razorpay/webhook` (signed).

12. Concurrency: what can collide, and what stops it

Every guarantee below is enforced by PostgreSQL (row locks, unique constraints, advisory locks), not by timing in the app, so it holds with many users and several server processes.

Race	What happens
Two flats book the last place in the same slot at the same moment	Booking runs in a transaction that first locks the amenity row (`SELECT … FOR UPDATE`). The second booking waits, then re-counts the slot and gets "already full". Weekly limits and closed days are checked inside the same lock
A booking cancelled twice, or cancelled while being changed	The booking row is locked; a cancelled booking cannot be refunded again
The checkout callback and the webhook arrive together for one payment	Both lock the payment order row; the second sees it is paid and returns the same receipt. `UNIQUE gateway_payment_id` backs it up
Two payments for one flat at once	The flat row is locked while interest is posted and the payment allocated, so postings never interleave
Two receipts get the same number	The year's counter row is incremented with `INSERT … ON CONFLICT DO UPDATE … RETURNING` in the same statement as the payment: the row lock serialises it, and a rolled-back payment releases its number with the transaction, so there are no gaps
A bill run retried after a crash, or issued twice	`UNIQUE (flat_id, period)` with `ON CONFLICT DO NOTHING`; one open run per month (a partial unique index); the run row is locked while approved
Two people upload a statement, or the same one twice	An advisory lock lets one import run at a time; rows whose UTR is already imported are marked duplicate

Two people resolve the same queued credit	The bank row and approval are locked; only one pending approval per row (a partial unique index); nobody approves their own request (a CHECK)
Two help-desk staff update one complaint	The complaint row is locked for the update, so events are written in order
Two outbox workers take the same messages	`FOR UPDATE SKIP LOCKED`: each worker claims a different batch; a batch left by a crashed worker is retried after 5 minutes
The gate tablet resends offline entries	Each entry carries a client id; `ON CONFLICT (client_id) DO NOTHING` keeps one
Two register imports at once	An advisory lock; nothing is saved until the preview is committed

13. Scaling

Where it stands. The brief's peak is about 8 requests a second (2,400 flats, most paying in the first 10 days). One process handles 270–300, with 95% of requests under 28 ms at 40 a second, so one township has about 30 times headroom. Heavy jobs are bounded: a bill run for every flat in 4.1 s, a 2,400-row statement in 17.5 s.

Why it scales out without a rewrite. The API is stateless: sessions, rate-limit counters and the outbox live in PostgreSQL, so any number of processes can run behind a load balancer (`TRUST_PROXY` makes rate limits see real addresses). Outbox workers share the queue with `SKIP LOCKED`; all locking is in the database (section 12), so correctness does not depend on running one process.

Growth	What I would do, in order
Busier single township	More app processes (horizontal); a bigger connection pool (`DB_POOL_MAX`) or PgBouncer; photos on S3 (already supported) behind a CDN
Years of data	The indexes are in place (migration 004). Next, partition `visitors`, `messages` and `ledger` by month and archive old gate entries; signed reports stay frozen
Heavy reports	Run the committee report and Tally export on a read replica; cache the month's figures once signed (they cannot change)
Many townships (a product)	Add a `township_id` to every table with row-level security, or one database per township for isolation; one deploy serves all
Bill-day spikes in alerts	The outbox already sends urgent messages first; more workers drain bulk alerts faster without delaying visitor alerts
Production hosting	The Terraform for AWS Mumbai (ECS, RDS with backups and a restore drill, S3) is written and validated in CI; Render's free plan is for the demo

What would break first: PostgreSQL write throughput for very large bill runs across many townships, and the single advisory lock for statement imports if many townships share a database. Both have the fixes above: batch inserts per township and a lock per township.

PART C · THINKING, QUESTIONS AND LIMITS

14. Cut, next and limits

- **Cut:** WhatsApp (needs Meta business approval); real SMS and payments (mocked, as the brief allows, following the real providers' steps, so switching is configuration); native apps; technician sign-in.
- **Next:** grow the labelled set from the help desk's corrections and compare Claude on it; remember UPI IDs by default, the biggest lever on automatic matching (family members' IDs are 10% of credits); WhatsApp

alerts through the same outbox; bank feeds instead of CSV uploads.

- **Limits:** the live app is on Render's free plan (it sleeps when idle, though a scheduled ping keeps it awake; photos uploaded there are lost on restart; the free database expires after 30 days); without a Claude key the classifier is keyword-based (81%, and it says when it is unsure); load-tested on one process.

15. Questions a reviewer might ask

Why not use an LLM for bank matching too?

A wrong match is a wrong flat's money and a committee's trust. Rules give the same answer every time, explain every point, and can be audited; the evaluation shows they post 55% automatically with no wrong posts. An LLM could suggest flats in the queue later, never post.

What happens when Claude is down, slow or wrong?

Down or slow (8 s): the local model answers and the complaint is saved. Wrong: the help desk sees the confidence and changes the category in one tap; the AI's guess is kept so accuracy can be measured. Dangerous: the safety rule has already escalated it before any model runs.

How do you know the AI is any good?

Section 7: 90 labelled complaints, scored on the same path a real complaint takes, re-run with `npm run eval` and held as floors in the tests. It found three real bugs, which were fixed.

Can two flats book the same last slot?

No. The booking locks the amenity row, re-counts the slot inside the lock and refuses the second (section 12). The same pattern protects payments, receipts, bill runs and approvals.

What if the same bank statement is uploaded twice, or a credit is a Razorpay settlement?

Rows whose UTR is already imported are marked duplicate and post nothing. Settlements are recognised and reconciled against payments already recorded in the app, so nothing is counted twice.

How is interest calculated, and why this way?

Simple interest on each bill's overdue principal at 21% a year, daily, posted before a payment so the payment clears interest first (A2). It follows the model bye-laws; the rate is a setting.

How can the committee trust the report?

Every rupee is a ledger line the database will not let anyone edit; money decisions need two people; receipts have no gaps; a signed month is frozen with a fingerprint checked on every view; `cli verify` proves all of it after a restore.

What if the gate tablet loses the network?

The guard keeps logging; entries wait on the tablet and sync with client ids, so retries never duplicate. Residents still get the alert once the entry reaches the server.

How does erasure (DPDP) square with keeping accounts?

Erasure closes the account, ends sessions and alerts, and deletes complaint photos and notes; bills, payments and receipts stay, because the society must keep its accounts by law.

Why a web app and not native apps?

The brief asks for an installable web app; it reaches every phone and desktop from one codebase with no store review, and gets push alerts and an offline shell. The Download app button covers every browser's way of installing.

Why one service, and what if it must scale?

One township needs a thirtieth of one process. It is stateless, with all locks in the database, so more processes can be added behind a load balancer without changing code (section 13).

What is mocked, and how hard is it to make real?

SMS, payments and photo storage. Each placeholder follows the real provider's steps (orders, signatures, webhooks), so adding MSG91, Razorpay or S3 credentials switches it on; no code changes.

Where did AI tools help, and where were the judgements mine?

AI drafted code, tests and copy faster and found gaps. The rules (matching thresholds, two-person checks, the safety rule above the model), the assumptions A1–A12, the evaluation design and the decision to measure rather than claim were judgements I made and can defend.